Tarea 2 — Redis + HBase: del stream a la historia

Procesamiento Masivo de Datos · ELE051-B / EIN102B · USM 2026-2 · Prof. Diego Ramírez
Integrantes: **Diego Peña y Benjamin Tello** · Entrega: 8 de noviembre de 2026

1. Arquitectura y reparto entre motores

El generador (semilla 2026) produce 864.000 eventos históricos y 7.200 en vivo. Los históricos van **directo a HBase** en batch. Los vivos entran a un stream de Redis (`sensores:t2`, MAXLEN ~10.000) y dos grupos independientes los consumen: **ingesta** los baja a HBase (XACK después del put) y **tablero** mantiene el "ahora" en Redis. La verificación cierra con los números de control: 871.200 eventos, 853.953 claves por tabla, XPENDING 0 en ambos grupos (**TODO CALZA**).

Dato	Motor	Estructura / clave	Por qué ahí y no en el otro
Última lectura de cada estación	Redis	hash `ultimo:<est>` (valor, ts, tipo)	Es el presente: O(1), sin disco. En HBase "lo último" exige un scan invertido por estación.
Top 5 de flujo	Redis	zset `ranking:flujo`	El ranking se mantiene al escribir; ordenar es O(log N). HBase tendría que leer todas las estaciones.
Ventana de 10 minutos	Redis	zset `ventana:<est>`, score = ts (ms)	Rango por tiempo y recorte O(log N + M); la memoria queda acotada por 10 min.
Catálogo de estaciones por tipo	Redis	set `tipo:<tipo>`	Membresía y listado instantáneos; es configuración, no historia.
Stream de eventos	Redis	stream `sensores:t2`	Consumer groups, replay y pendientes (XPENDING/XAUTOCLAIM).
Historia por estación	HBase	`lecturas`: estación#ts	5 días no caben en memoria; la clave ordena por estación y tiempo y permite rangos por región.
Historia por tipo	HBase	`lecturas_tipo`: tipo#ts#estación	P4b resuelve todas las estaciones de un tipo con un solo scan.
Versiones y auditoría	HBase	celdas con `VERSIONS = 3`	Permite ver reprocesamientos y el "último por llegada" sin perder el dato.

2. Parte C — Preguntas

P1. "Ahora": Redis contra HBase

Redis leyó las 16 estaciones y el top 5 en **2,3 ms** (16 HGETALL + ZREVRANGE). HBase necesitó **205,4 ms** con 16 scans invertidos (uno por estación, `reverse=True, limit=1`): es **89× más lento**. Ambos motores devuelven los mismos valores porque ambos usan el último por medición. En HBase "lo último" no sale gratis: la clave estación#ts ordena ascendente, así que hay que recorrer cada estación al revés (o mantener un índice aparte).

La estación que cambia de respuesta es **TAL-001**: por medición su último valor es **41,0** a las 00:59:40.500; por llegada es **48,9** a las 00:58:48.335 (una lectura tardía escrita al final de la hora en vivo). Un dashboard usaría Redis: microsegundos y "ahora" es su contrato; HBase sirve para auditar qué llegó último, no para el tablero.

P2. Ventana de 10 minutos (Redis)

Se leyeron solo los `ventana:*`, con el ancla en la medición más reciente del stream (`tablero:max_ts`): **3,3 ms** para las 8 estaciones de flujo. El zset usa el timestamp como score porque insertar es O(log N), `ZRANGEBYSCORE` responde la ventana en O(log N + M) sin recorrer la historia, y `ZREMRANGEBYSCORE` recorta lo viejo en el mismo orden. Sin recorte, 5 días de flujo (~432.000 entradas) ocuparían **~15,3 MB** (35 B/entrada medidos con MEMORY USAGE) y en producción crecería sin techo; hoy las 8 claves suman 19.944 B.

P3. Un rango en HBase (CC-001, 04-nov 08:00–10:00)

Hay **891 lecturas**, promedio **20,55**; los promedios por cuarto de hora van de 15,61 (08:00) a 25,87 (08:45). El scan por rango (row_start/row_stop sobre `CC-001#2026-11-04T08:00…`) tarda **17,4 ms** y toca una sola región. El recorrido de la tabla completa con `PrefixFilter` tarda **600,8 ms**: es **34,5× más lento** y ambos devuelven lo mismo. Con 5 años en vez de 5 días el rango seguiría acotado (~2 h de una estación) mientras el recorrido completo crecería ~365× (~219 s por corrida). Por eso la row key empieza por estación.

P4. Dos patrones de acceso: catálogo Redis + HBase

El catálogo `tipo:calidad_aire` devolvió 4 estaciones (CC-004, CHI-001, COR-001, TAL-003). Entre 07:00 y 09:00 del 05-nov: CC-004 promedio 68,09 / máx 80,0 a las 07:22:15; CHI-001 62,18 / 80,0 a las 07:07:14; COR-001 22,78 / 52,3 a las 08:18:32; TAL-003 48,14 / 76,5 a las 07:27:43. El patrón (a) con un rango por estación en `lecturas` tarda **60,1 ms**; el (b) con un solo scan en `lecturas_tipo` tarda **40,2 ms** (1,5× más rápido con solo 4 estaciones; la brecha crece con el catálogo). La segunda tabla cuesta ~871.200 escrituras extra y **125 MB** vs **72 MB** en disco (tras flush). Si falla el segundo put, el XACK no ocurre y el lote se reprocesa: no se pierde, pero entre ambos puts hay una ventana de inconsistencia. No valdría la pena si las consultas por tipo fueran raras y el catálogo enorme.

P5. Hotspotting y salting

La hora en vivo se escribió en tres tablas. **hv_estacion** (est#ts, 16 regiones) repartió 409–478 escrituras por región (max/min 1,17). **hv_ts** (ts#est, 5 regiones) dejó **7.048 de 7.048 escrituras en una sola región**: hotspot puro, y esa región crece con el tiempo. **hv_salt** (salt#ts#est, 4 regiones, crc32 mod 4) repartió 1.280–2.219 (max/min 1,73). La consulta 00:10–00:20 del 07-nov devolvió 1.180 filas con los tres diseños: **99,2 ms** (16 rangos), **19,4 ms** (1 rango) y **33,9 ms** (4 rangos). Para el patrón mixto del pipeline (escrituras continuas + consultas por estación) elegiría **est#ts**: reparte las escrituras sin hotspot y mantiene P3 en una región, a cambio de 99,2 ms en la consulta de todas las estaciones. Si esa consulta dominara y las escrituras fueran bajas, elegiría **ts#est** (19,4 ms) aceptando el hotspot; **salt** es el compromiso: 33,9 ms y 4 regiones. Cambiaría a salt si las escrituras saturaran una región.

P6. Pregunta propia: ahora (Redis) contra la historia (HBase)

¿Cuánto se desvían las 5 estaciones de flujo con más flujo ahora respecto de su media y desviación histórica de 5 días? Redis decide el "qué" y el "ahora" (`ranking:flujo` + `ultimo:*`); HBase decide el "cuándo" y la escala (scan por estación para media y desviación). Resultado en **4,5 s**: SPP-002 z=0,39; HUA-002 **z=2,23 (anómala)**; HUA-001 z=-0,35; PEN-001 z=-0,30; CC-002 z=-0,03. No es una variante de P1: allí se comparan últimos valores entre motores; acá el valor actual se contrasta con la distribución de 5 días. Solo Redis obligaría a mantener 5 días de estadísticas en memoria; solo HBase obligaría a definir "ahora" sin el ranking ya listo.

3. Parte D — Cuando algo se cae

D.1 El consumidor muere entre el put y el XACK

Se reinició el grupo `ingesta` y se corrió `consumidor_ingesta.py --matar-en-lote 1`: escribió el primer lote de 200 entradas y llamó `os._exit(1)` antes del XACK. Evidencia:

- XPENDING quedó con **200 entradas** a nombre de **ingesta-1** (ids 1794149975931-0 a 1794149980908-0).
- Un segundo consumidor las recuperó con **XAUTOCLAIM** (`--recuperar`), las reescribió y siguió con las 7.000 restantes. XPENDING final = **0**.
- HBase terminó con **853.953 filas en cada tabla**, exactamente el número de control: ni faltan ni sobran.
- La celda `CC-003#2026-11-07T00:00:00.000-03:00` quedó con **3 versiones** del mismo valor (la carga original, el consumidor muerto y la recuperación), todas con `value=53.9` y distinto timestamp de escritura.

Reprocesar no duplica filas porque la row key sale de la estación y del milisegundo de medición, no del instante de llegada: se escribe la misma celda y HBase agrega una versión (`VERSIONS = 3`), no una fila. El XACK va después del put justamente para que una muerte así deje el lote pendiente y otro consumidor pueda recuperarlo. Si se invirtiera el orden, esos 200 eventos se perderían.

D.2 Redis se reinicia: AOF con volumen vs sin persistencia

Se levantaron dos Redis efímeros con la misma carga (3.000 eventos, grupo con 300 pendientes, `ultimo:*`, ranking y ventana) y se reiniciaron con `docker restart`. Con la configuración elegida (**AOF + appendfsync everysec + volumen**) sobrevivió todo: stream (3.000), grupo, 300 pendientes, `ultimo:CC-001`, `ranking:flujo` (2) y `ventana:CC-001` (5). Sin persistencia (`--appendonly no --save ""`) no sobrevivió **nada**: el stream, los grupos, los pendientes y el tablero desaparecieron.

Ventana de pérdida: lo que está en Redis y todavía no en HBase es el lag del grupo `ingesta` más sus pendientes (normalmente < 1 lote de 200 eventos). Con AOF everysec se pierde como máximo ~1 s de escrituras; con RDB, minutos; sin persistencia, todo. La historia ya escrita en HBase está a salvo en cualquiera de los casos. En un sistema real aceptaría perder a lo sumo 1 s de eventos no confirmados: `appendfsync everysec`. Con `always` la pérdida sería cero pero cada XADD pagaría un fsync.

4. Tiempos, hardware y escala

Medición	Resultado	Nota
Carga histórica (864.000 eventos, batch 2.000)	44,7 s · 19.348 ev/s	18.965 filas/s en 2 tablas (37.931 mutaciones/s)
Creación de tablas con SPLITS	33,6 s	16 regiones en lecturas, 13 en lecturas_tipo
Hora en vivo (7.200 eventos)	180,0 s · 40 ev/s	25 ms por evento = 20× el ritmo real
P1 Redis / HBase	2,3 ms / 205,4 ms	89× (16 HGETALL vs 16 scans invertidos)
P2 ventana Redis	3,3 ms	8 ZRANGEBYSCORE; 19.944 B
P3 rango / tabla completa	17,4 ms / 600,8 ms	34,5×; 891 lecturas
P4 por estación / por tipo	60,1 ms / 40,2 ms	4 estaciones de calidad del aire; 125 MB vs 72 MB
P5 est#ts / ts#est / salt	99,2 / 19,4 / 33,9 ms	1.180 filas en los tres casos

P6 ahora + historia	4,5 s	5 estaciones × 5 días de scan

Hardware: Intel Core i5-12500H (12 núcleos / 16 hilos), 16 GB RAM, Windows 11, Docker Desktop con 4 GB asignados; HBase 2.5.15 y Redis 7 en contenedores locales; cliente Python en el host. Medianas de 5 corridas descartando la primera.

Honestidad con la escala: HBase standalone en un PC no es un cluster. Los tiempos absolutos de P3 y P5 no extrapolan a un cluster con regiones en máquinas distintas: lo que sí extrapola es el diseño (16 vs 1 vs 4 regiones, hotspot de ts#est) y la diferencia O(rango) vs O(tabla). Las conclusiones de Redis sí son representativas de una instancia única, no de Redis Cluster.

5. Qué no funcionó y qué haríamos distinto

- **Happybase 1.3 no permite crear tablas con split keys** (el Thrift1 de HBase no expone SPLITS). Se resuelve con `hbase shell` dentro del contenedor; funciona, pero acopla la creación de tablas al nombre del servicio en el compose.
- **Las versiones de celda no se ven desde happybase** (colapsa columnas repetidas en un dict). Para D.1 se usa `hbase shell get ... VERSIONS => 3`.
- El primer intento de medir P4 comparó tamaños en disco antes de un flush: el MemStore hacía ver `lecturas` cientos de veces más chica. Se corrigió forzando `flush` antes de `du`; la lección es que en HBase "tamaño en disco" no es el tamaño real hasta que se vacía el MemStore.
- La ventana se recorta solo cuando avanza la medición más reciente del stream: si el stream se detiene, las ventanas quedan congeladas. Es lo correcto para "ahora", pero significa que no hay un TTL independiente.
- Si se repitiera la tarea, separaría la creación de tablas del script de carga y usaría un cliente Thrift2/Admin para los SPLITS, sin depender de `hbase shell`.

6. Conclusiones

- **El motor se elige por la pregunta, no por el dato.** Redis gana todo lo que empieza por "ahora" (P1: 89×, P2: 3 ms) porque mantiene estructuras ya ordenadas; HBase gana todo lo que empieza por "historia" y tiene un rango (P3: 34,5× sobre el recorrido completo).
- **La idempotencia es la pieza que hace barato el at-least-once.** Row key por estación y medición + XACK después del put permiten reprocesar sin perder ni duplicar (D.1), incluso si el consumidor muere a mitad de lote.
- **La segunda tabla no es gratis:** duplica las escrituras y ocupa 72 MB extra en este dataset; se justifica cuando el patrón "por tipo" es frecuente (P4), no cuando es raro.
- **La elección de row key decide el hotspot:** `ts#est` concentró el 100% de las escrituras en una región. El salting lo reparte, pero encarece la consulta por estación. El diseño correcto depende del patrón dominante, no de una regla universal.
- **Lo que no sobrevive a una caída hay que poder reconstruir:** con AOF se pierde a lo sumo 1 s; sin persistencia se pierde el presente completo, pero la historia en HBase sigue intacta. Esa separación es justamente la que hace tolerable la falla.